

## Introduction

In this exercise we will solve the Cart-Pole problem using on-policy SARSA, off-policy Q learning, and off-policy SARSA with an  $\epsilon$ -greedy policy.

```
[ ]: from abc import ABC, abstractmethod
import gymnasium as gym
import numpy as np

[2]: ENV_ID, SEED = "CartPole-v1", 10
NUM_TIMESTEPS_GOAL = 10000
env = gym.make(ENV_ID, max_episode_steps=NUM_TIMESTEPS_GOAL)

[3]: # helpers

def _get_even_int(n, min_val=6):
    n = int(n)
    if n < min_val:
        raise ValueError(f"Cannot specify number smaller than {min_val}")
    return n+1 if n % 2 else n # return even number

def _discretize(n, interval):
    l,u = interval
    mid = n // 2
    lower = np.linspace(l, 0, mid, endpoint=False)
    upper = np.linspace(0, u, mid+1)
    bins = np.concatenate([lower,upper])
    return bins

def neat_int(arr):
    return [int(x) for x in arr]
```

The algorithms we are going to implement are all temporal differencing methods and are very similar to each other, so we will define a base class. TD methods are tabular methods and therefore we must discretize the state space in a similar way to Assignment 2 (except now the terminal values will be included in our discrete space). Our MC control algorithms failed to converge in Assignment 2, so this time we have decided to substantially increase the size of the statespace by increasing the number of bins from  $10^4 = 10000$  to  $15^4 = 50625$ . The loops do take substantially longer to finish as a result.

Aside from this, the basic concepts of the discretization essentially remain the same but with one important difference: The terminal states are now included in the episodes, and therefore must be included in our discrete space. In Assignment 2, the bins were indexed by 0 (first bin was bin 0, second bin was bin 1,...). We have changed this; The first bin is bin 1, second bin is bin 2, and so on. Any terminal value is represented by bin 0 (for each of the state variables). So  $n = 14$  bins refer to the  $n = 14$  partitions of the non-terminal values, however in effect the size of our state space is  $(n+1)^4 = 15^4$  when accounting for the terminal states.

Also note that in the `init` method, behaviour is left as "False". This value determines whether the control method will use the target policy or a behaviour policy to generate the episode. This way, any base classes that use on-policy methods remain unaffected, and off-policy methods will work without the need to copy-paste the entire chunk of code for a trivial change.

```
[ ]: class TDAgent(ABC):
```

```

def __init__(self, gamma=0.9, alpha=0.2, epsilon=0.15,
              num_bins=14, vel_range=(-2.5,2.5), angular_vel_range=(-3.5,3.5)):
    self.gamma = gamma
    self.alpha = alpha
    self.epsilon = epsilon
    self.MAX_ITERATIONS = 100000
    self.discretize_spaces(num_bins, vel_range, angular_vel_range)
    self.Q = [] # temporary placeholder value

    # this way we dont have to copy paste the entire control
    # method just for Agent C
    self.behaviour = None

@abstractmethod
def update_rule(self):
    pass

@abstractmethod
def select_action(self, state, behaviour=False):
    pass

def discretize_spaces(self, num_bins, vel_range, angular_vel_range):
    self.n = _get_even_int(num_bins)
    self.n_plus = self.n + 1
    self.table_dims = (self.n_plus, self.n_plus, self.n_plus, self.n_plus, 2)
    pos_range = (-2.4, 2.4) # non terminal range for position
    angle_range = (-0.2095, 0.2095) # non terminal range for angle (radians)
    intervals = [pos_range, vel_range, angle_range, angular_vel_range]
    self.discrete_space = [_discretize(self.n, x) for x in intervals]

def get_discrete(self, state):
    def mapper(i):
        s, bins = state[i], self.discrete_space[i]
        if s == bins[-1]:
            return self.n
        elif s > bins[-1]:
            return 0
        bin_index = np.digitize(s, bins)
        return bin_index
    return tuple(map(mapper, range(len(state))))

def get_greedy_action(self, state):
    return np.argmax(self.Q[state])

def set_terminals_to_zero(self):
    j = np.arange(self.n_plus)
    self.Q[0,j,j,j,:] = 0
    self.Q[j,0,j,j,:] = 0
    self.Q[j,j,0,j,:] = 0
    self.Q[j,j,j,0,:] = 0

```

```

def control(self, env):

    self.Q = np.random.random_sample(self.table_dims)
    self.set_terminals_to_zero()

    output_logs = []
    success = False

    for num_iter in range(1, self.MAX_ITERATIONS+1):

        state, _ = env.reset()
        state = self.get_discrete(state)
        reward, finished = None, False

        count = 0
        while not finished:
            count += 1
            s = state
            action = self.select_action(state, self.behaviour)

            state, reward, terminated, truncated, _ = env.step(action)
            state = self.get_discrete(state)

            a,r,s_prime = action, reward, state
            a_prime = self.select_action(state, self.behaviour)

            curr = self.Q[s][a]
            update = self.update_rule(s,a,r,s_prime,a_prime)
            new_val = self.alpha*(update) + curr
            self.Q[s][a] = new_val

            if truncated:
                success == True
                break
            if terminated:
                finished = True

        if not num_iter % 5000:
            print(f"Iteration number: {num_iter}. Duration of episode: {count}␣
↳timesteps. \n\n")

        env.close()
        if success:
            break

    if success:
        print(f"Success! The agent was able to balance the pole for at least␣
↳{NUM_TIMESTEPS_GOAL} timesteps.")
    else:
        print(f"Failure to meet goal after {self.MAX_ITERATIONS} iterations.")

```

```
return output_logs
```

## Part A: On-Policy SARSA

Agent A inherits from the base TD class above and implements an on-policy SARSA algorithm:

```
[5]: class AgentA(TDAgent):

    def __init__(self, gamma=0.9, alpha=0.2, epsilon=0.15,
                 num_bins=14, vel_range=(-2.5,2.5), angular_vel_range=(-3.5,3.5)):

        super().__init__(gamma,alpha,epsilon,num_bins,vel_range,angular_vel_range)

    def update_rule(self, s, a, r, s_prime, a_prime):
        return self.gamma*np.max(self.Q[s_prime]) - self.Q[s][a] + r

    def select_action(self, state, behaviour=False):
        greedy = self.get_greedy_action(state)
        non_greedy = greedy ^ 1
        if np.random.random_sample() <= self.epsilon:
            return non_greedy
        return greedy

[6]: agent_A = AgentA()
      agent_A.control(env)
```

Iteration number: 5000. Duration of episode: 74 timesteps.

Iteration number: 10000. Duration of episode: 154 timesteps.

Iteration number: 15000. Duration of episode: 178 timesteps.

Iteration number: 20000. Duration of episode: 105 timesteps.

Iteration number: 25000. Duration of episode: 100 timesteps.

Iteration number: 30000. Duration of episode: 154 timesteps.

Iteration number: 35000. Duration of episode: 115 timesteps.

Iteration number: 40000. Duration of episode: 144 timesteps.

Iteration number: 45000. Duration of episode: 150 timesteps.

Iteration number: 50000. Duration of episode: 134 timesteps.

Iteration number: 55000. Duration of episode: 156 timesteps.

Iteration number: 60000. Duration of episode: 218 timesteps.

Iteration number: 65000. Duration of episode: 154 timesteps.

Iteration number: 70000. Duration of episode: 367 timesteps.

Iteration number: 75000. Duration of episode: 143 timesteps.

Iteration number: 80000. Duration of episode: 358 timesteps.

Iteration number: 85000. Duration of episode: 239 timesteps.

Iteration number: 90000. Duration of episode: 169 timesteps.

Iteration number: 95000. Duration of episode: 171 timesteps.

Iteration number: 100000. Duration of episode: 233 timesteps.

Failure to meet goal after 100000 iterations.

[6]: []

There results are far more promising than the ones from Assignment 2. We are consistently seeing episodes longer than 100 timesteps. This is partially due to the efficacy of TD algorithms (and learning online concurrently with simulation), and partially due to the increased state space.

## Part B: Off-Policy Q-learning

The Agent B inherits from Agent A (same control method), and all we need to do is tweak the update rule.

```
[7]: class AgentB(AgentA):  
  
    def __init__(self, gamma=0.9, alpha=0.2, epsilon=0.15,  
                 num_bins=14, vel_range=(-2.5,2.5), angular_vel_range=(-3.5,3.5)):  
  
        super().__init__(gamma,alpha,epsilon,num_bins,vel_range,angular_vel_range)  
  
    def update_rule(self, s, a, r, s_prime, a_prime):  
        return self.gamma*self.Q[s_prime][a_prime] - self.Q[s][a] + r
```

```
[9]: agent_B = AgentB()  
agent_B.control(env)
```

Iteration number: 5000. Duration of episode: 48 timesteps.

Iteration number: 10000. Duration of episode: 138 timesteps.

Iteration number: 15000. Duration of episode: 144 timesteps.

Iteration number: 20000. Duration of episode: 191 timesteps.

Iteration number: 25000. Duration of episode: 230 timesteps.

Iteration number: 30000. Duration of episode: 176 timesteps.

Iteration number: 35000. Duration of episode: 146 timesteps.

Iteration number: 40000. Duration of episode: 145 timesteps.

Iteration number: 45000. Duration of episode: 158 timesteps.

Iteration number: 50000. Duration of episode: 230 timesteps.

Iteration number: 55000. Duration of episode: 140 timesteps.

Iteration number: 60000. Duration of episode: 150 timesteps.

Iteration number: 65000. Duration of episode: 130 timesteps.

Iteration number: 70000. Duration of episode: 281 timesteps.

Iteration number: 75000. Duration of episode: 196 timesteps.

Iteration number: 80000. Duration of episode: 188 timesteps.

Iteration number: 85000. Duration of episode: 52 timesteps.

Iteration number: 90000. Duration of episode: 209 timesteps.

Iteration number: 95000. Duration of episode: 369 timesteps.

Iteration number: 100000. Duration of episode: 317 timesteps.

Failure to meet goal after 100000 iterations.

[9]: []



## Part C: Off-Policy Expected SARSA

Agent C requires a little more tweaking, and as such, it inherits from TDAgent rather than one of the other two classes. Here, the target policy is chosen to be an epsilon greedy policy even though this is an off policy method. The reason this is done is so that the update rule can take the expected value of the next possible state-action pairs (which is meaningless with a deterministic policy). Since our target policy is  $\epsilon$ -greedy with  $\epsilon = 0.15$ , we chose a similarly  $\epsilon$ -greedy behaviour policy which is more exploratory ( $\epsilon_b = \epsilon * 2 = 0.3$ ). The requirement of coverage is still clearly met.

```
[12]: class AgentC(TDAgent):

    def __init__(self, gamma=0.9, alpha=0.2, epsilon=0.15,
                 num_bins=14, vel_range=(-2.5,2.5), angular_vel_range=(-3.5,3.5)):

        super().__init__(gamma,alpha,epsilon,num_bins,vel_range,angular_vel_range)
        self.behaviour = True
        self.b_epsilon = epsilon*2 # epsilon for behaviour

    def select_action(self, state, behavior=False):
        greedy = self.get_greedy_action(state)
        non_greedy = greedy ^ 1
        threshold = self.b_epsilon if behavior else self.epsilon
        if np.random.random_sample() <= threshold:
            return non_greedy
        return greedy

    def update_rule(self, s, a, r, s_prime, a_prime):
        greedy_val = np.max(self.Q[s_prime])
        non_greedy_val = np.min(self.Q[s_prime])
        expected_val_est = greedy_val*(1-self.epsilon) + non_greedy_val*self.epsilon
        return self.gamma*expected_val_est - self.Q[s][a] + r

[13]: agent_C = AgentC()
agent_C.control(env)
```

Iteration number: 5000. Duration of episode: 83 timesteps.

Iteration number: 10000. Duration of episode: 140 timesteps.

Iteration number: 15000. Duration of episode: 118 timesteps.

Iteration number: 20000. Duration of episode: 19 timesteps.

Iteration number: 25000. Duration of episode: 48 timesteps.

Iteration number: 30000. Duration of episode: 85 timesteps.

Iteration number: 35000. Duration of episode: 50 timesteps.

Iteration number: 40000. Duration of episode: 86 timesteps.

Iteration number: 45000. Duration of episode: 215 timesteps.

Iteration number: 50000. Duration of episode: 28 timesteps.

Iteration number: 55000. Duration of episode: 94 timesteps.

Iteration number: 60000. Duration of episode: 106 timesteps.

Iteration number: 65000. Duration of episode: 117 timesteps.

Iteration number: 70000. Duration of episode: 86 timesteps.

Iteration number: 75000. Duration of episode: 24 timesteps.

Iteration number: 80000. Duration of episode: 32 timesteps.

Iteration number: 85000. Duration of episode: 147 timesteps.

Iteration number: 90000. Duration of episode: 102 timesteps.

Iteration number: 95000. Duration of episode: 71 timesteps.

Iteration number: 100000. Duration of episode: 169 timesteps.

Failure to meet goal after 100000 iterations.

[13]: []

## Discussion

TD Algorithms are far more suited to this problem than naive MC-based control techniques. These algorithms show promising results for convergence (though they are still quite inefficient and will take a long time to reach that point, if ever). Agents A and B seem to perform better than Agent C. However, Agent C took less than half as much the time to finish the loop as the others. This is consistent with our theoretical

knowledge: Agent C uses an epsilon greedy policy, and not a true optimal policy (but this allows us to use expected value in the update step, which provides drastic improvement in computational efficiency).